

Cross-Validation, Ridge Regression, and the Lasso

Teaching Team of STA314

Fall 2026

Learning objectives

By the end of this tutorial, you should be able to:

1. explain why training error is not a reliable estimate of test error;
2. implement the validation-set approach and k -fold cross-validation in R;
3. use cross-validation to compare models of different complexity;
4. fit ridge and lasso regression models using `glmnet`; and
5. use cross-validation to select the tuning parameter λ .

Setup

This tutorial uses the `Auto` and `Hitters` datasets from the `ISLR2` package and the `glmnet` package for ridge and lasso regression.

If necessary, install the packages once by running:

```
install.packages(c("ISLR2", "glmnet"))
```

```
library(ISLR2)
library(glmnet)
```

Cross-validation

We begin by modelling fuel efficiency, measured by `mpg`, using the `Auto` dataset. Our immediate goal is to compare polynomial regression models that use `horsepower` to predict `mpg`.

```
data("Auto")
```

```
dim(Auto)
```

```
## [1] 392  9
```

```
head(Auto)
```

```
##   mpg cylinders displacement horsepower weight acceleration year origin
## 1   18         8         307         130   3504          12.0    70     1
## 2   15         8         350         165   3693          11.5    70     1
## 3   18         8         318         150   3436          11.0    70     1
## 4   16         8         304         150   3433          12.0    70     1
## 5   17         8         302         140   3449          10.5    70     1
## 6   15         8         429         198   4341          10.0    70     1
##                                     name
## 1 chevrolet chevelle malibu
```

```
## 2      buick skylark 320
## 3      plymouth satellite
## 4      amc rebel sst
## 5      ford torino
## 6      ford galaxie 500
```

```
summary(Auto$mpg)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##      9.00   17.00   22.75   23.45   29.00   46.60
```

Why not use training error?

If we evaluate a model on the same observations used to fit it, a more flexible model will generally have a smaller residual sum of squares and training MSE. This does not guarantee that it will predict new observations well.

We therefore estimate test error using observations that were not used to fit the model.

The validation-set approach

The validation-set approach randomly divides the observations into two groups:

- a **training set**, used to fit the model; and
- a **validation set**, used to estimate its prediction error.

The split below places half of the observations in each set.

```
set.seed(442)

n <- nrow(Auto)
train_id <- sample(seq_len(n), size = floor(n / 2))

Auto_train <- Auto[train_id, ]
Auto_valid <- Auto[-train_id, ]

nrow(Auto_train)

## [1] 196
nrow(Auto_valid)

## [1] 196
```

We first fit a simple linear regression using `horsepower` as the only predictor.

```
fit_linear <- lm(mpg ~ horsepower, data = Auto_train)

valid_pred <- predict(fit_linear, newdata = Auto_valid)
valid_mse <- mean((Auto_valid$mpg - valid_pred)^2)

valid_mse

## [1] 25.65745
```

To compare polynomial models efficiently, we create a function that fits a polynomial of a specified degree and returns its validation MSE.

```
validation_mse <- function(degree, train_data, valid_data) {
  fit <- lm(
```

```

    mpg ~ poly(horsepower, degree = degree, raw = TRUE),
    data = train_data
  )

  predictions <- predict(fit, newdata = valid_data)
  mean((valid_data$mpg - predictions)^2)
}

degrees <- 1:5

validation_results <- data.frame(
  degree = degrees,
  validation_MSE = sapply(
    degrees,
    validation_mse,
    train_data = Auto_train,
    valid_data = Auto_valid
  )
)

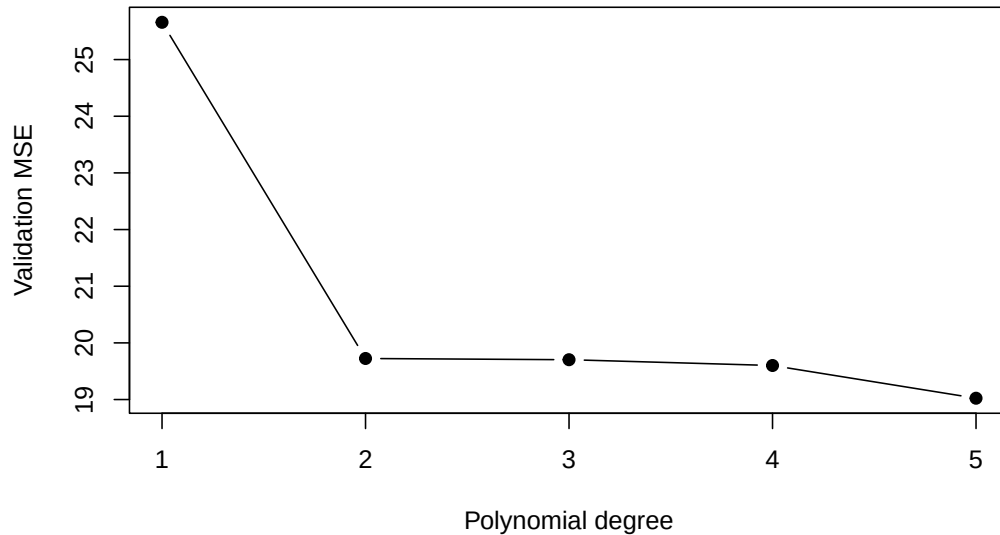
validation_results

##   degree validation_MSE
## 1      1      25.65745
## 2      2      19.72442
## 3      3      19.70286
## 4      4      19.60053
## 5      5      19.02321

plot(
  validation_results$degree,
  validation_results$validation_MSE,
  type = "b",
  pch = 19,
  xlab = "Polynomial degree",
  ylab = "Validation MSE",
  main = "Validation error by model complexity"
)

```

Validation error by model complexity



The quadratic model will often improve substantially on the linear model, while additional polynomial terms may provide little improvement.

Sensitivity to the split

A limitation of the validation-set approach is that the result depends on which observations happen to enter the validation set. We can repeat the procedure using several random splits.

```
validation_by_seed <- function(seed, degrees = 1:5) {  
  set.seed(seed)  
  train_id <- sample(seq_len(nrow(Auto)), size = floor(nrow(Auto) / 2))  
  
  train_data <- Auto[train_id, ]  
  valid_data <- Auto[-train_id, ]  
  
  sapply(  
    degrees,  
    validation_mse,  
    train_data = train_data,  
    valid_data = valid_data  
  )  
}  
  
split_results <- sapply(c(1, 2, 3, 4, 5), validation_by_seed)  
rownames(split_results) <- paste0("Degree_", 1:5)  
colnames(split_results) <- paste0("Seed_", 1:5)  
  
round(split_results, 2)
```

```
##           Seed_1 Seed_2 Seed_3 Seed_4 Seed_5  
## Degree_1  23.27  25.73  21.99  24.32  20.89  
## Degree_2  18.72  20.43  17.70  19.03  16.80  
## Degree_3  18.79  20.39  17.71  19.03  17.03  
## Degree_4  19.16  20.31  17.63  19.29  17.31  
## Degree_5  19.41  19.80  17.10  19.13  16.76
```

The estimates vary because each model is evaluated on only one validation set. This motivates k -fold cross-validation.

k -fold cross-validation

In k -fold cross-validation:

1. randomly divide the data into k approximately equal-sized folds;
2. hold out one fold and fit the model using the other $k - 1$ folds;
3. calculate the prediction error on the held-out fold;
4. repeat until every fold has served as the validation fold; and
5. average the k validation errors.

For squared-error loss, the estimate is

$$CV_{(k)} = \frac{1}{n} \sum_{i=1}^n \left(y_i - \hat{f}^{(-v(i))}(x_i) \right)^2,$$

where $v(i)$ identifies the fold containing observation i , and $\hat{f}^{(-v(i))}$ is fitted without that fold.

The function below implements k -fold CV directly. Writing out the loop makes clear that each observation is used for validation exactly once.

```
kfold_mse <- function(data, degree, k = 10, seed = 442) {  
  set.seed(seed)  
  
  # Randomly assign observations to approximately equal-sized folds.  
  fold_id <- sample(rep(1:k, length.out = nrow(data)))  
  squared_errors <- numeric(nrow(data))  
  
  for (fold in 1:k) {  
    train_data <- data[fold_id != fold, ]  
    valid_data <- data[fold_id == fold, ]  
  
    fit <- lm(  
      mpg ~ poly(horsepower, degree = degree, raw = TRUE),  
      data = train_data  
    )  
  
    predictions <- predict(fit, newdata = valid_data)  
    squared_errors[fold_id == fold] <- (valid_data$mpg - predictions)^2  
  }  
  
  mean(squared_errors)  
}
```

We now use the **same folds for every polynomial degree**, making the model comparison fair.

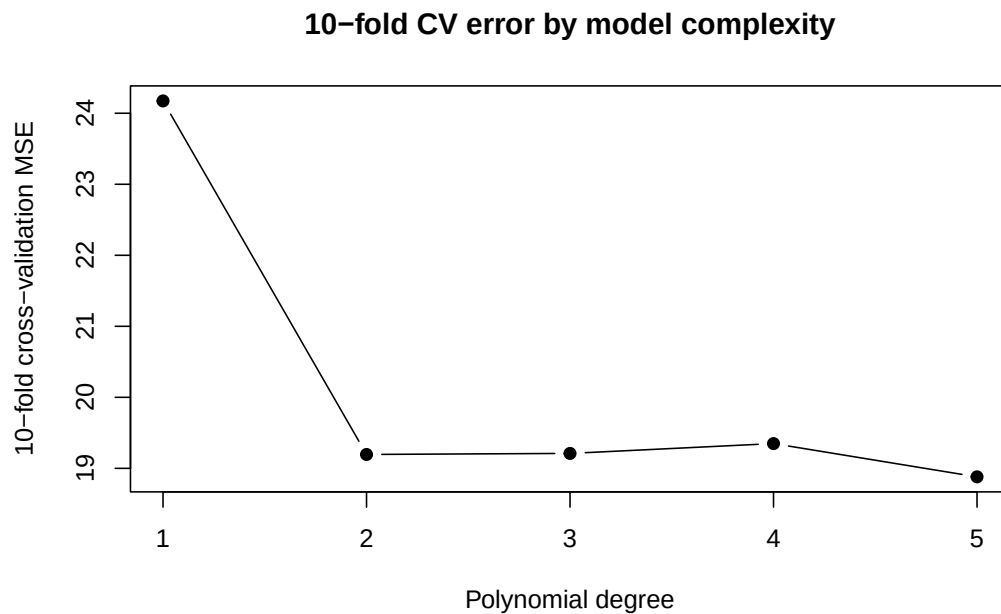
```
degrees <- 1:5  
  
cv_results <- data.frame(  
  degree = degrees,  
  CV_MSE = sapply(  
    degrees,  
    function(d) kfold_mse(Auto, degree = d, k = 10, seed = 442)  
  )  
)
```

```
)

cv_results

##   degree  CV_MSE
## 1      1 24.17339
## 2      2 19.19541
## 3      3 19.21011
## 4      4 19.34862
## 5      5 18.88027

plot(
  cv_results$degree,
  cv_results$CV_MSE,
  type = "b",
  pch = 19,
  xlab = "Polynomial degree",
  ylab = "10-fold cross-validation MSE",
  main = "10-fold CV error by model complexity"
)
```



Which degree would you choose?

Leave-one-out cross-validation

Leave-one-out cross-validation (LOOCV) is the special case $k = n$. Each validation fold contains one observation.

```
loocv_results <- data.frame(
  degree = degrees,
  LOOCV_MSE = sapply(
    degrees,
    function(d) kfold_mse(
      Auto,
      degree = d,
      k = nrow(Auto),
```

```

    seed = 442
  )
)
)

loocv_results

##    degree LOOCV_MSE
## 1      1  24.23151
## 2      2  19.24821
## 3      3  19.33498
## 4      4  19.42443
## 5      5  19.03321

```

LOOCV uses nearly all observations to fit each model, but it requires fitting the model n times. Ten-fold CV usually gives a similar conclusion at a lower computational cost.

What happens after cross-validation?

Cross-validation is used to **choose the model**, not to obtain a final set of coefficients. Once we choose the polynomial degree, we refit that model using the full dataset.

```

chosen_degree <- cv_results$degree[which.min(cv_results$CV_MSE)]

final_auto_model <- lm(
  mpg ~ poly(horsepower, degree = chosen_degree, raw = TRUE),
  data = Auto
)

chosen_degree

## [1] 5

coef(final_auto_model)

##                (Intercept)
##                -3.223056e+01
## poly(horsepower, degree = chosen_degree, raw = TRUE)1
##                3.700109e+00
## poly(horsepower, degree = chosen_degree, raw = TRUE)2
##                -7.142408e-02
## poly(horsepower, degree = chosen_degree, raw = TRUE)3
##                5.931086e-04
## poly(horsepower, degree = chosen_degree, raw = TRUE)4
##                -2.281076e-06
## poly(horsepower, degree = chosen_degree, raw = TRUE)5
##                3.329553e-09

```

We do **not** average the coefficient estimates from the k fitted models. Those fits were temporary and were used to estimate validation error.

Ridge regression and the lasso

We now apply cross-validation to choose the tuning parameter for two regularized regression methods.

Let $\mathbf{y}$ be the response vector, $\mathbf{X}$ the design matrix, and $\boldsymbol{\beta}$ the coefficient vector. Ordinary least squares minimizes

$$\text{RSS} = \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p x_{ij} \beta_j \right)^2.$$

OLS can be unstable when predictors are highly correlated, when p is large relative to n , or when an overly flexible model has high variance. Ridge and lasso regression add a penalty to shrink the coefficient estimates toward zero.

The intercept β_0 is typically **not penalized**.

Ridge regression

Ridge regression solves

$$\underset{\beta_0, \boldsymbol{\beta}}{\text{minimize}} \left\{ \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p x_{ij} \beta_j \right)^2 + \lambda \sum_{j=1}^p \beta_j^2 \right\}.$$

The L_2 penalty shrinks coefficients smoothly toward zero, but generally does not set them exactly equal to zero. Ridge is often useful when many predictors contribute to the response or when predictors are highly correlated.

The lasso

The lasso solves

$$\underset{\beta_0, \boldsymbol{\beta}}{\text{minimize}} \left\{ \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p x_{ij} \beta_j \right)^2 + \lambda \sum_{j=1}^p |\beta_j| \right\}.$$

The L_1 penalty can set some coefficient estimates exactly equal to zero. The lasso therefore performs shrinkage and variable selection simultaneously.

The role of λ

- When $\lambda = 0$, both methods reduce to OLS.
- As λ increases, the coefficients are shrunk more strongly toward zero.
- More shrinkage generally increases bias but decreases variance.
- We use cross-validation to choose λ based on estimated prediction error.

Because the penalty acts directly on coefficient size, predictors should be placed on comparable scales. By default, `glmnet()` standardizes predictors before fitting the model and returns coefficients on their original scales.

Preparing the Hitters data

We use the `Hitters` dataset to predict a baseball player's salary from measures of performance and experience.

```
data("Hitters")  
  
dim(Hitters)
```

```
## [1] 322 20
```

```
head(Hitters)
```

```
##           AtBat Hits HmRun Runs RBI Walks Years CAtBat CHits CHmRun
## -Andy Allanson    293   66    1  30  29   14    1    293   66    1
## -Alan Ashby       315   81    7  24  38   39   14   3449   835   69
## -Alvin Davis      479  130   18  66  72   76    3   1624   457   63
## -Andre Dawson     496  141   20  65  78   37   11   5628  1575  225
## -Andres Galarra... 321   87   10  39  42   30    2    396   101   12
## -Alfredo Griffin  594  169    4  74  51   35   11   4408  1133   19
##           CRuns CRBI CWalks League Division PutOuts Assists Errors
## -Andy Allanson     30   29   14      A      E    446    33    20
## -Alan Ashby       321  414   375      N      W    632    43    10
## -Alvin Davis      224  266   263      A      W    880    82    14
## -Andre Dawson     828  838   354      N      E    200    11     3
## -Andres Galarra...  48   46    33      N      E   805    40     4
## -Alfredo Griffin  501  336   194      A      W   282   421    25
##           Salary NewLeague
## -Andy Allanson      NA      A
## -Alan Ashby       475.0      N
## -Alvin Davis      480.0      A
## -Andre Dawson     500.0      N
## -Andres Galarra...  91.5      N
## -Alfredo Griffin  750.0      A
```

Some players have missing salary values. We remove those observations before fitting the models.

```
Hitters <- na.omit(Hitters)
```

```
# glmnet requires a numeric predictor matrix.
# model.matrix() automatically creates dummy variables for factors.
x <- model.matrix(Salary ~ ., data = Hitters)[, -1]
y <- Hitters$Salary
```

```
dim(x)
```

```
## [1] 263 19
```

```
head(x)
```

```
##           AtBat Hits HmRun Runs RBI Walks Years CAtBat CHits CHmRun
## -Alan Ashby       315   81    7  24  38   39   14   3449   835   69
## -Alvin Davis      479  130   18  66  72   76    3   1624   457   63
## -Andre Dawson     496  141   20  65  78   37   11   5628  1575  225
## -Andres Galarra... 321   87   10  39  42   30    2    396   101   12
## -Alfredo Griffin  594  169    4  74  51   35   11   4408  1133   19
## -Al Newman       185   37    1  23   8   21    2    214    42    1
##           CRuns CRBI CWalks LeagueN DivisionW PutOuts Assists Errors
## -Alan Ashby       321  414   375      1      1    632    43    10
## -Alvin Davis      224  266   263      0      1    880    82    14
## -Andre Dawson     828  838   354      1      0    200    11     3
## -Andres Galarra...  48   46    33      1      0    805    40     4
## -Alfredo Griffin  501  336   194      0      1    282   421    25
## -Al Newman       30    9    24      1      0     76   127     7
##           NewLeagueN
## -Alan Ashby         1
```

```
## -Alvin Davis          0
## -Andre Dawson         1
## -Andres Galarrraga    1
## -Alfredo Griffin      0
## -Al Newman            0
```

We remove the first column of `model.matrix()` because it is the intercept column. `glmnet()` fits an unpenalized intercept automatically.

Ridge regression in R

In `glmnet()`, `alpha = 0` fits ridge regression and `alpha = 1` fits the lasso. These uses of `alpha` should not be confused with a hypothesis-test significance level.

We first fit ridge regression over a grid of λ values selected by `glmnet()`.

```
ridge_fit <- glmnet(
  x = x,
  y = y,
  alpha = 0,
  standardize = TRUE
)
```

```
ridge_fit
```

```
##
## Call:  glmnet(x = x, y = y, alpha = 0, standardize = TRUE)
##
##      Df  %Dev Lambda
## 1   19  0.00 255300
## 2   19  1.16 232600
## 3   19  1.27 211900
## 4   19  1.39 193100
## 5   19  1.53 176000
## 6   19  1.67 160300
## 7   19  1.83 146100
## 8   19  2.00 133100
## 9   19  2.19 121300
## 10  19  2.39 110500
## 11  19  2.61 100700
## 12  19  2.85  91740
## 13  19  3.12  83590
## 14  19  3.40  76170
## 15  19  3.71  69400
## 16  19  4.05  63240
## 17  19  4.41  57620
## 18  19  4.81  52500
## 19  19  5.23  47840
## 20  19  5.69  43590
## 21  19  6.19  39710
## 22  19  6.72  36190
## 23  19  7.29  32970
## 24  19  7.90  30040
## 25  19  8.55  27370
## 26  19  9.24  24940
```

##	27	19	9.98	22730
##	28	19	10.77	20710
##	29	19	11.60	18870
##	30	19	12.47	17190
##	31	19	13.39	15660
##	32	19	14.35	14270
##	33	19	15.36	13000
##	34	19	16.40	11850
##	35	19	17.48	10800
##	36	19	18.60	9837
##	37	19	19.74	8963
##	38	19	20.91	8167
##	39	19	22.10	7442
##	40	19	23.30	6781
##	41	19	24.51	6178
##	42	19	25.72	5629
##	43	19	26.92	5129
##	44	19	28.12	4674
##	45	19	29.29	4258
##	46	19	30.45	3880
##	47	19	31.57	3535
##	48	19	32.66	3221
##	49	19	33.71	2935
##	50	19	34.71	2674
##	51	19	35.68	2437
##	52	19	36.59	2220
##	53	19	37.46	2023
##	54	19	38.27	1843
##	55	19	39.04	1680
##	56	19	39.76	1530
##	57	19	40.43	1394
##	58	19	41.06	1271
##	59	19	41.64	1158
##	60	19	42.18	1055
##	61	19	42.69	961
##	62	19	43.16	876
##	63	19	43.59	798
##	64	19	44.00	727
##	65	19	44.37	662
##	66	19	44.73	604
##	67	19	45.06	550
##	68	19	45.37	501
##	69	19	45.66	457
##	70	19	45.93	416
##	71	19	46.19	379
##	72	19	46.43	345
##	73	19	46.67	315
##	74	19	46.89	287
##	75	19	47.10	261
##	76	19	47.31	238
##	77	19	47.51	217
##	78	19	47.71	198
##	79	19	47.89	180
##	80	19	48.08	164

```
## 81 19 48.26 150
## 82 19 48.45 136
## 83 19 48.63 124
## 84 19 48.80 113
## 85 19 48.98 103
## 86 19 49.16 94
## 87 19 49.34 86
## 88 19 49.52 78
## 89 19 49.70 71
## 90 19 49.88 65
## 91 19 50.06 59
## 92 19 50.24 54
## 93 19 50.42 49
## 94 19 50.60 45
## 95 19 50.77 41
## 96 19 50.95 37
## 97 19 51.13 34
## 98 19 51.30 31
## 99 19 51.48 28
## 100 19 51.64 26
```

Each column of the fitted coefficient matrix corresponds to one value of λ .

```
dim(coef(ridge_fit))
```

```
## [1] 20 100
```

```
# First few lambda values considered by glmnet
```

```
head(ridge_fit$lambda)
```

```
## [1] 255282.1 232603.5 211939.7 193111.5 175956.0 160324.6
```

```
# Coefficients at one value along the path
```

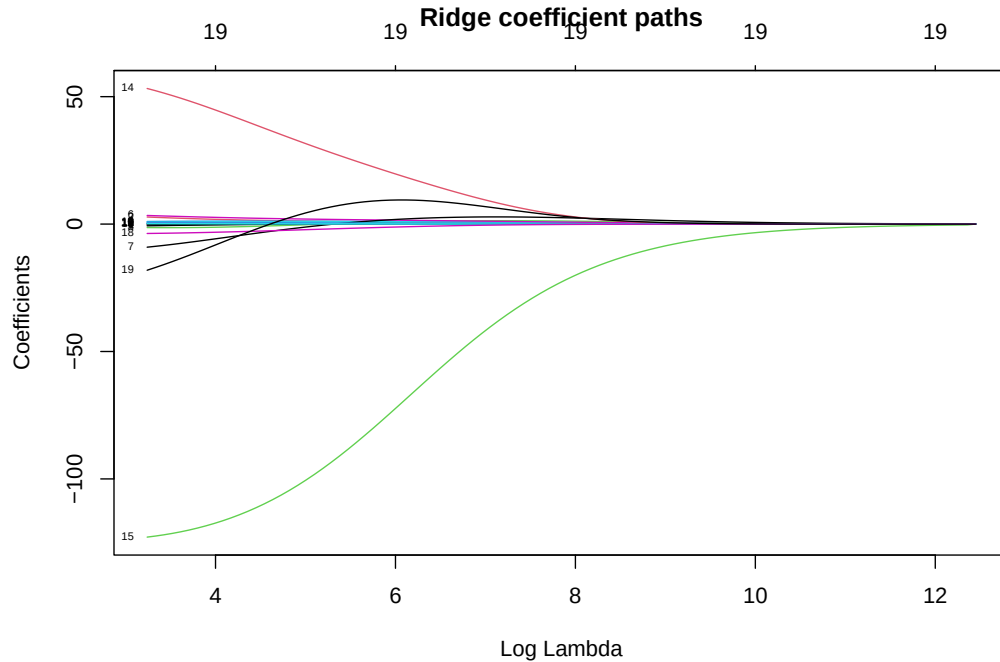
```
coef(ridge_fit, s = ridge_fit$lambda[50])
```

```
## 20 x 1 sparse Matrix of class "dgCMatrix"
```

```
##              s1
## (Intercept) 213.066443434
## AtBat      0.090095728
## Hits       0.371252756
## HmRun      1.180126956
## Runs       0.596298287
## RBI        0.594502390
## Walks      0.772525466
## Years      2.473494238
## CAtBat     0.007597952
## CHits      0.029272172
## CHmRun     0.217335716
## CRuns      0.058705097
## CRBI       0.060722036
## CWalks     0.058698830
## LeagueN    3.276567828
## DivisionW -21.889942619
## PutOuts    0.052667119
## Assists    0.007463678
## Errors     -0.145121336
## NewLeagueN 2.972759126
```

The coefficient path shows how the estimates change with λ .

```
plot(
  ridge_fit,
  xvar = "lambda",
  label = TRUE,
  main = "Ridge coefficient paths"
)
```



Moving toward larger $\log(\lambda)$ values applies a stronger penalty, so all coefficient estimates move closer to zero. Ridge generally retains every predictor.

Selecting λ using k -fold CV

For each candidate λ , `cv.glmnet()` performs the following steps:

1. fit the ridge model on $k - 1$ folds;
2. predict the held-out fold;
3. repeat until every fold has been held out;
4. average the held-out squared errors; and
5. compare the mean CV error across the candidate λ values.

We explicitly create the fold assignments so ridge and lasso use the same folds.

```
set.seed(442)
K <- 10
fold_id <- sample(rep(1:K, length.out = nrow(x)))
table(fold_id)
```

```
## fold_id
##  1  2  3  4  5  6  7  8  9 10
## 27 27 27 26 26 26 26 26 26 26
```

```
cv_ridge <- cv.glmnet(
  x = x,
  y = y,
```

```

alpha = 0,
foldid = fold_id,
type.measure = "mse",
standardize = TRUE
)

```

```
cv_ridge$lambda.min
```

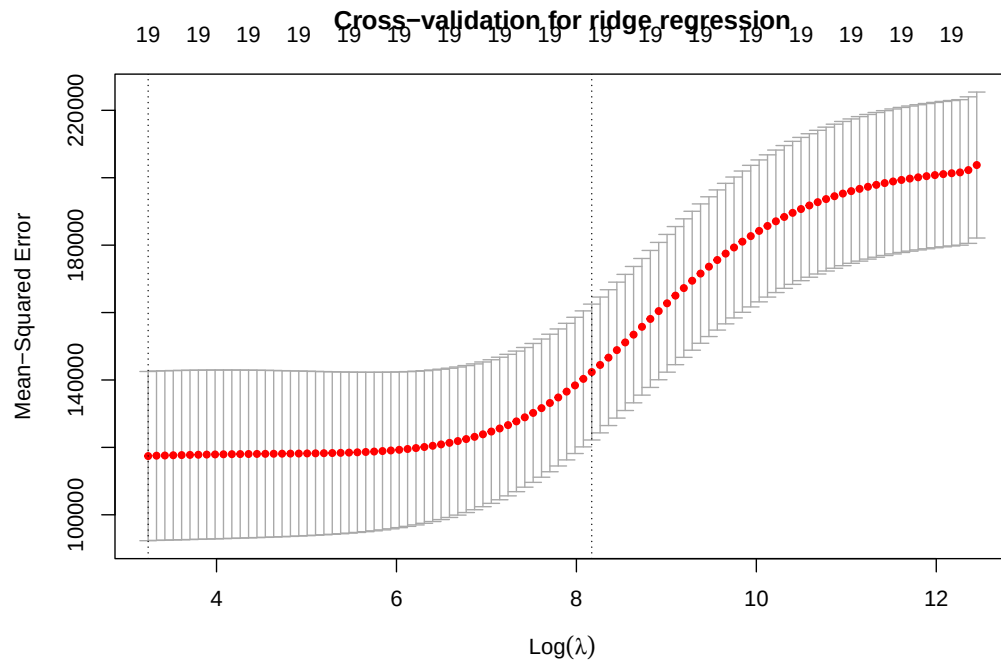
```
## [1] 25.52821
```

```
cv_ridge$lambda.1se
```

```
## [1] 3535.367
```

`lambda.min` is the value with the smallest estimated CV error. `lambda.1se` is the largest value whose CV error is within one standard error of the minimum; it applies more shrinkage and may be preferred when a simpler or more stable model is desired.

```
plot(cv_ridge, main = "Cross-validation for ridge regression")
```



The dashed vertical lines identify `lambda.min` and `lambda.1se`.

```

ridge_coef_min <- coef(cv_ridge, s = "lambda.min")
ridge_coef_1se <- coef(cv_ridge, s = "lambda.1se")

```

```
ridge_coef_min
```

```

## 20 x 1 sparse Matrix of class "dgCMatrix"
##              s1
## (Intercept)  8.112693e+01
## AtBat        -6.815959e-01
## Hits         2.772312e+00
## HmRun        -1.365680e+00
## Runs         1.014826e+00
## RBI          7.130224e-01

```

```
## Walks      3.378558e+00
## Years     -9.066800e+00
## CAtBat    -1.199478e-03
## CHits      1.361029e-01
## CHmRun     6.979958e-01
## CRuns      2.958896e-01
## CRBI       2.570711e-01
## CWalks    -2.789666e-01
## LeagueN    5.321272e+01
## DivisionW -1.228345e+02
## PutOuts    2.638876e-01
## Assists    1.698796e-01
## Errors    -3.685645e+00
## NewLeagueN -1.810510e+01
```

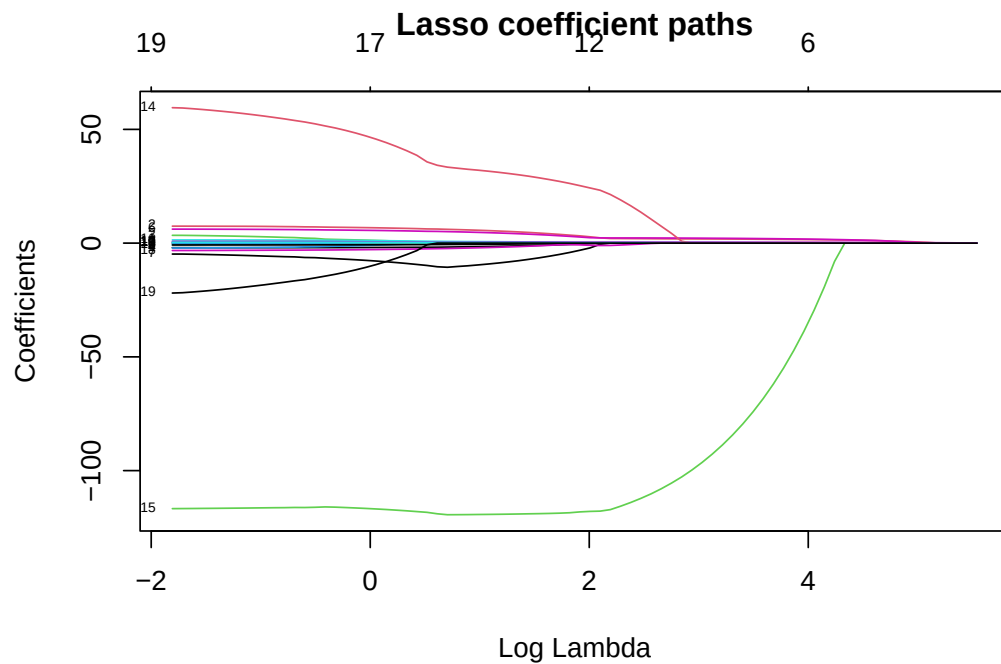
The model stored by `cv.glmnet()` has already been refitted to the full dataset along the complete λ sequence. Calling `coef()` at the chosen λ therefore returns the final coefficient estimates; we do not average coefficients across folds.

The lasso in R

We now repeat the analysis using `alpha = 1`.

```
lasso_fit <- glmnet(
  x = x,
  y = y,
  alpha = 1,
  standardize = TRUE
)

plot(
  lasso_fit,
  xvar = "lambda",
  label = TRUE,
  main = "Lasso coefficient paths"
)
```



For a sufficiently large λ , all lasso coefficients are zero. As λ decreases, predictors enter the model.

```
cv_lasso <- cv.glmnet(
  x = x,
  y = y,
  alpha = 1,
  foldid = fold_id,
  type.measure = "mse",
  standardize = TRUE
)
```

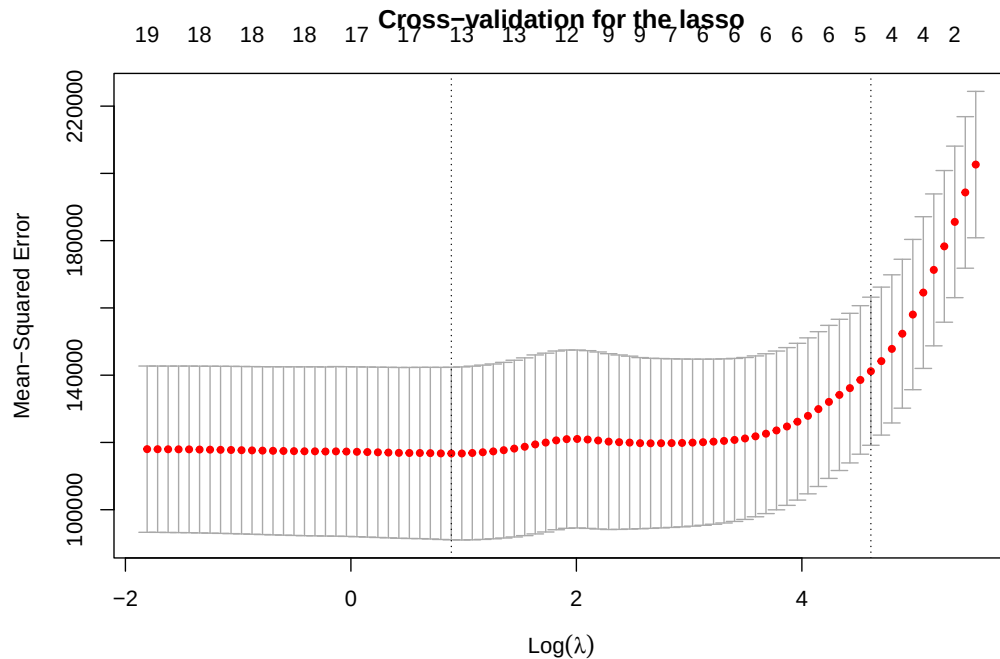
```
cv_lasso$lambda.min
```

```
## [1] 2.436791
```

```
cv_lasso$lambda.1se
```

```
## [1] 100.6885
```

```
plot(cv_lasso, main = "Cross-validation for the lasso")
```



```
lasso_coef_min <- coef(cv_lasso, s = "lambda.min")
lasso_coef_1se <- coef(cv_lasso, s = "lambda.1se")

lasso_coef_min
```

```
## 20 x 1 sparse Matrix of class "dgCMatrix"
##              s1
## (Intercept) 129.415571
## AtBat      -1.6130155
## Hits       5.8058915
## HmRun      .
## Runs       .
## RBI        .
## Walks      4.8469340
## Years     -9.9724045
## CAtBat     .
## CHits      .
## CHmRun     0.5374550
## CRuns      0.6811938
## CRBI       0.3903563
## CWalks    -0.5560144
## LeagueN   32.4646094
## DivisionW -119.3480842
## PutOuts    0.2741895
## Assists    0.1855978
## Errors    -2.1650837
## NewLeagueN .
```

We can identify the predictors selected by the lasso at `lambda.min`.

```
lasso_coef_vector <- as.vector(lasso_coef_min)
names(lasso_coef_vector) <- rownames(lasso_coef_min)

selected_predictors <- names(lasso_coef_vector)[
```

```

lasso_coef_vector != 0 & names(lasso_coef_vector) != "(Intercept)"
]

removed_predictors <- names(lasso_coef_vector)[
  lasso_coef_vector == 0 & names(lasso_coef_vector) != "(Intercept)"
]

selected_predictors

## [1] "AtBat"      "Hits"      "Walks"     "Years"     "CHmRun"    "CRuns"
## [7] "CRBI"      "CWalks"    "LeagueN"   "DivisionW" "PutOuts"   "Assists"
## [13] "Errors"

removed_predictors

## [1] "HmRun"      "Runs"      "RBI"       "CAtBat"    "CHits"
## [6] "NewLeagueN"

length(selected_predictors)

## [1] 13

```

Comparing ridge and lasso

The minimum cross-validated MSE for each method is stored in its `cvm` vector.

```

comparison <- data.frame(
  method = c("Ridge", "Lasso"),
  lambda_min = c(cv_ridge$lambda.min, cv_lasso$lambda.min),
  minimum_CV_MSE = c(min(cv_ridge$cvm), min(cv_lasso$cvm)),
  nonzero_predictors = c(
    sum(as.vector(coef(cv_ridge, s = "lambda.min"))[-1] != 0),
    sum(as.vector(coef(cv_lasso, s = "lambda.min"))[-1] != 0)
  )
)

comparison

##   method lambda_min minimum_CV_MSE nonzero_predictors
## 1  Ridge  25.528210         117418                19
## 2  Lasso   2.436791         116710                13

```

Because the methods use different penalty definitions, their numerical λ values should not be compared directly. Compare their cross-validated errors and model properties instead.

In general:

- **Ridge** is attractive when many predictors carry signal, especially when predictors are correlated.
- **Lasso** is attractive when a sparse, more interpretable model is plausible.
- Neither method is universally better; cross-validation helps assess predictive performance for the data at hand.

Optional: estimating final test error

The CV error used to choose λ is a model-selection estimate. If we want an especially clean, final estimate of generalization performance, we can set aside a test set before tuning. The test set must not be used to

choose the method or λ .

```
set.seed(442)
test_id <- sample(seq_len(nrow(x)), size = floor(0.25 * nrow(x)))

x_train <- x[-test_id, ]
y_train <- y[-test_id]
x_test  <- x[test_id, ]
y_test  <- y[test_id]

# Create folds using the training observations only.
train_fold_id <- sample(rep(1:10, length.out = nrow(x_train)))

cv_lasso_train <- cv.glmnet(
  x_train,
  y_train,
  alpha = 1,
  foldid = train_fold_id,
  type.measure = "mse"
)

test_predictions <- predict(
  cv_lasso_train,
  newx = x_test,
  s = "lambda.min"
)

test_mse <- mean((y_test - as.vector(test_predictions))^2)
test_mse
```

Summary

- A single validation split is simple, but its estimated error can depend strongly on the random split.
- k -fold CV averages performance across several held-out folds and generally gives a more stable model comparison.
- After using CV to choose a model, refit that model on all available training data; do not average the fold-specific coefficients.
- Ridge uses an L_2 penalty and usually retains all predictors.
- The lasso uses an L_1 penalty and can set some coefficients exactly to zero.
- The penalty parameter λ controls the bias–variance trade-off and is commonly selected using cross-validation.

Discussion questions

1. Why might the validation-set estimate change when we change the random seed?
2. Why should the same folds be used when comparing several candidate models?
3. Why is standardization especially important for ridge and lasso regression?
4. What is the practical difference between `lambda.min` and `lambda.1se`?
5. If ridge has a slightly smaller CV error but lasso uses far fewer predictors, which model would you choose? What additional context would affect your decision?